This full-text paper was peer-reviewed at the direction of IEEE Instrumentation and Measurement Society prior to the acceptance and publication.

A Deployable Edge Computing Solution for Machine Condition Monitoring

Xijia Zhao
Department of Electrical and Computer Engineering
University of Kentucky
Lexington, USA
Xijia.Zhao@uky.edu

Peng Wang
Department of Electrical and Computer Engineering
Department of Mechanical and Aerospace Engineering
University of Kentucky
Lexington, USA
Edward.Wang@uky.edu

2024 IEEE International Instrumentation and Measurement Technology Conference (I2MTC) | 979-8-3503-8090-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/I2MTC60896.2024.10560795

***Abstract*—Machine condition monitoring and predictive maintenance are critical components of smart manufacturing. The realization of machine predictive maintenance relies on synthetic integration of advanced sensing, Internet of Things (IoT), cloud computing, and Artificial Intelligence (AI)-enabled data analytics for machine condition diagnosis and prognosis. One challenge that hurdles the manufacturing shop floor from implementing predictive maintenance is its outdated hardware and limited internet bandwidth, which prevents the floor from transmitting all collected data to the cloud for centralized data analytics. One promising solution is edge-cloud computing and decision-making architecture. In such an infrastructure, collected sensing data will be first processed on edge to directly obtain machine diagnosis results (e.g., normal vs. abnormal), and only extracted features or selected raw data will be transmitted to the cloud for further analytics (e.g., remaining life prognosis). To improve the affordability and applicability of edge computing, this study proposed a novel and deployable edge device that adopts a feature-based Tiny Machine Learning (Tiny ML) model to achieve vibration data processing and machine condition monitoring while maintaining the modeling generalizability and computing efficiency. It integrates a Microelectromechanical Systems (MEMS) accelerometer for vibration data sampling (up to 20 kHz) and a microcontroller for edge computing and communication to the cloud. The integral IoT system, which involves the edge device, cloud services, and wireless communication between the edge and cloud is presented, emphasizing its plug-and-play capability. In this framework, MQTT protocol is adopted for communication among edge, cloud, and end users, allowing processed signal information and machine diagnosis results to be published to end users in real-time. Extensive experiments have been conducted in the context of industrial motor condition monitoring and fault diagnosis. High classification accuracy has been demonstrated by the device-deployable Tiny ML, comparable or even better than the integration of high-cost commercial sensors with more advanced ML analytics. Additionally, the edge device's durability and power efficiency are also evaluated.**

Keywords—Edge computing, Tiny ML, machine condition monitoring, fault diagnosis

I. INTRODUCTION

Industry 4.0 is transforming conventional manufacturing shop floor to smart factories, which are driven by advanced technologies such as the Internet of Things (IoT), cyber-physical systems, Machine Learning (ML), Artificial Intelligence (AI), big data analytics, digital twins, etc. [1]. By integrating these techniques, manufacturing systems can be greatly enhanced, achieving reliability with real-time machine health monitoring and prognosis, efficiency by optimizing processes for improved cycling time and quality control, and sustainability by reducing scrap rates and material waste. A representative application of this integration is machine prognosis and predictive maintenance, which encompasses real-time machine condition monitoring, diagnosis of abnormal conditions, and prognosis of machine remaining useful life [2] to proactively schedule maintenance actions toward reduced maintenance downtime and maintenance costs. Achieving condition monitoring relies on two elements: sensors (e.g., vibration, current) and sensing data analytics.

A conventional way of realizing predictive maintenance on the shop floor is through cloud computing, namely transmitting all collected sensor data from local machines to the cloud and performing data analytics and decision making on the cloud. Due to the large number of sensors employed to cover the entire shop floor which results in high demands for power and cloud sources for data transmission, and outdated hardware (i.e., limited internet bandwidth) [3], implementing cloud computing and centralized decision-making is not applicable for most manufacturers. Even though the data eventually migrates to the cloud, extensive storage and computing resources required for data management and analysis lead to increased costs beyond capital investment.

Edge computing emerges as a promising solution for handling big data. Since its introduction in 2004, edge computing has been attracting attention from academia and industry, because of its cost-effective computing power, streamlined data processing pipeline, and good scalability compared to cloud computing [4]. It has been widely investigated in various fields, such as process monitoring [5], quality control [6], and machine predictive maintenance [7]. In the area of predictive maintenance, edge computing can fulfill the four fundamental elements of machine condition monitoring on a single chip: data acquisition, data preprocessing, feature extraction, and pattern recognition [8]. Data acquisition and preprocessing related studies adapt conventional data filtering, enhancement, and compression methods to edge implementation [9-11], while feature extraction and pattern recognition related research seek to implement advanced data analytics with improved reliability considering edge device computing constraints [12-15]. In the context of machine pattern recognition, inspired by the successful studies of AI and ML for fault diagnosis [16], researchers started exploring the application of Tiny Machine Learning (Tiny ML) for data/feature pattern recognition on the edge. Chen et al. [17] developed an edge device to harvest

Authorized licensed use limited to: FUNDACAO UNIVERSIDADE DO RIO GRANDE. Downloaded on June 26,2026 at 15:54:37 UTC from IEEE Xplore. Restrictions apply.

energy from vibration, the outputs from which were analyzed by several Tiny ML methods for spectrum analysis. Morenas et al. [18] applied edge computing to analyze motor current signals through decision trees, random forests, and support vector machines. Magadánet et al. [19] introduced an edge/fog computing architecture to analyze vibration signals on edge devices and gateways through feature-based pattern recognition. While these studies have demonstrated successes in some application contexts, the solutions are typically unable to continuously monitor machine conditions and analyze high-frequency sensing data. A fully deployable and reliable end-to-end data collection, processing, and decision-making pipeline still lacks.

This paper presents a systematic and practical solution of edge computing for machine condition monitoring in industrial settings, which involves a novel and low-cost edge device employing Tiny ML techniques, cloud services, and communication approaches between components. The major novelties and scientific contributions include:

- **Low-cost and compact design, high-frequency sampling:** A compact edge device that integrates a Microelectromechanical Systems (MEMS) accelerometer with a microcontroller on a single board is programmed to achieve a sampling and transmitting rate of up to 20 kHz. The cost of a unit is under $30.
- **Efficient and reliable hardware and algorithms:** A computationally efficient and generalizable feature-based Tiny ML model is employed on the device for machine condition monitoring. And performance of the device (i.e., accuracy of fault diagnosis, power consumption) is demonstrated on actual induction motors.
- **Plug-and-play system, end-to-end decision-making:** The solution provides an IoT framework that integrates the edge device and cloud services, making it ready to be deployed in industrial settings. It is facilitated by lightweight wireless communication and user-friendly interfaces for device operation, end-to-end information publication, and machine condition visualization toward improved machine health management.

The rest of this paper is organized as follows: Section II introduces the developed edge device, communication protocols, and user interfaces. Sections III and IV experimentally evaluate the performance of the developed device and system, in terms of vibration monitoring, fault diagnosis accuracies, and power consumption, which is followed by the conclusion in Section V.

II. System Development

This system is developed following a typical IoT architecture [20], encompassing all stages from hardware design and programming to cloud applications for data analytics and user visualization. This architecture can be categorized into four layers, as illustrated in Fig.1. The base of this structure is the hardware layer, whose components are employed to capture the vibration signal. Then edge computing such as signal feature extraction and Tiny ML model inference are applied on the device application layer. Data on the edge device (e.g., diagnosis outcomes, extracted features, selected raw data) are optionally transmitted to the cloud through the network layer, depending on the specific objectives. The cloud application layer provides users with functionalities for model training and displaying information.

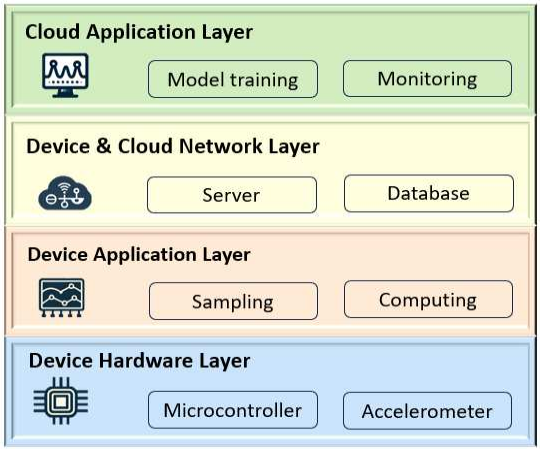


Fig. 1. The 4-layer system architecture, from the hardware layer to the cloud application layer.

A. Device hardware layer

An overview of the edge device is shown in Fig.2. For vibration monitoring, MEMS accelerometers are commonly adopted because of their high sensitivity, cost-effectiveness, and strong reliability [21]. It is connected to a microcontroller (MCU), which is critical for facilitating communication between the sensor and the cloud, as well as performing computing tasks on the device. For this purpose, ESP32 WROOM 32E is selected as the MCU, which is a dual-core 32-bit microprocessor equipped with an onboard antenna enabling it to achieve a communication rate of up to 150 Mbps through 2.4 GHz Wi-Fi under the 802.11 b/g/n standards. It boasts a 448 KB ROM and 520 KB RAM, thereby fulfilling the storage requirements for edge computing tasks. It is also able to handle communication with the MEMS accelerometer with various interfaces such as Analog-Digital Converter (ADC), Intern IC Bus (I2C), and Serial Peripheral Interface (SPI). SPI is chosen as the communication method for its high communication speed over other protocols, which can achieve up to 10 Mb/s [22].

There are several auxiliary circuits on the board, such as a rechargeable lithium battery for power supply and a USB-C port for charging and downloading programs.

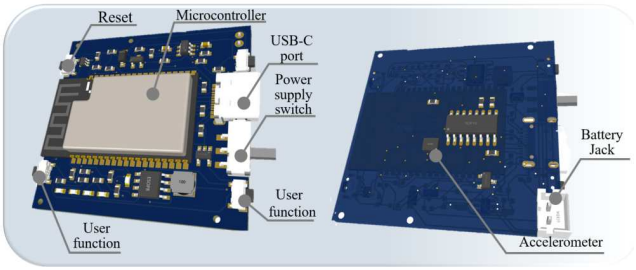


Fig. 2. Illustration of the developed edge device.

B. Device application layer

This layer focuses on embedding specific algorithms within the MCU, to drive the execution of two tasks: data transmission and edge computing (e.g., feature extracting, Tiny ML model inferring).

In the first case, the challenge lies in continuous and high-frequency data sampling of the vibration signal. Tasks on a single core of the embedded device are executed sequentially, which means data acquisition must halt during wireless data transmission to the cloud, leading to signal loss as depicted in

Authorized licensed use limited to: FUNDACAO UNIVERSIDADE DO RIO GRANDE. Downloaded on June 26,2026 at 15:54:37 UTC from IEEE Xplore. Restrictions apply.

Fig.3(a). To address this issue, FreeRTOS [23], a multi-threaded real-time operating system for microcontrollers, is introduced. It enables the execution of multi-threaded tasks, specifically, data sampling and sending, on the dual cores of ESP32. As illustrated in Fig.3(b), once Core#0 completes the sampling task of a data chunk, the data is transferred to a shared physical memory for easy access and rapid retrieval by Core#1 within 100 instruction cycles on the CPU. Core#1 is then triggered to send the data to the server while Core#0 proceeds with the next sampling task. This method ensures the integrity (i.e., no data loss) of the sampled signal, provided the sampling time (t_s) is greater than the sending time (t_m).

In executing the second task, i.e., vibration signal feature extraction and Tiny ML inference, the MCU acts as a fault detector. Considering the computing and RAM (520 KB) limits, it is difficult to deploy advanced ML algorithms with

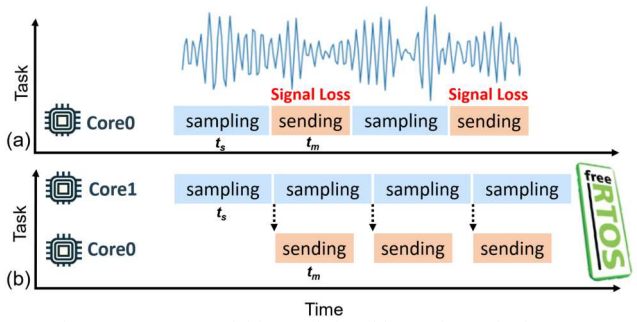


Fig. 3. (a) Data acquisition and signal loss using a single-core microcontroller and (b) the dual-core structure using freeRTOS.

complex structures and numerous tunable parameters. In this study, a conventional but effective and efficient feature-based pattern recognition tiny ML is adopted. It analyses the horizontal and vertical vibration signal by extracting 9 time-domain features, i.e., Mean, Root Mean Square (RMS), Standard Deviation (SD), Skewness, Kurtosis, Crest Factor, Latitude Factor, Shape Factor, Impulse Factor, and 7 frequency-domain features, i.e., Mean Frequency (MF), Average Frequency (AF), Stabilization Factor of Waveshape (SFW), Coefficient of Variability (CV), Frequency-domain Skewness, Frequency-domain Kurtosis (FK), Root Mean Square Ratio (RMSR), which have been validated to reduce the computational complexity while maintaining its effectiveness in detecting motor faults [24]. Two computationally efficient algorithms are designed for time-domain and frequency-domain feature extraction in C++, respectively. The complexity of algorithms is crucial for implementation on the edge device due to the limited storage and capacity of ESP32.

In the algorithm for time-domain feature extraction, the input data is only traversed once, so the time complexity is ***O(m*n)***, where ***n*** is the length of each signal window, and it iterates ***m*** times for ***m*** different windows. However, the frequency-domain feature extraction is relatively time-consuming. All the frequency-domain features are based on the power spectrum density, which is obtained by Welch's method. In Welch's method, a signal sequence is divided into ***m*** overlapped subsets of the length ***n***, and the windowed discrete Fourier transform (DFT) for each subset is calculated. The DFT calculation dominates the overall time complexity. Each subset is processed in $O(n^2)$ and the signal that contains ***m*** segments is processed in $O(m*n^2)$. To reduce the time complexity, the fast Fourier transform (FFT), which has a time complexity of ***O(n*log n)*** for a single signal with ***n*** data points[25], is adopted, and it helps to reduce the time complexity to ***O(m*n*log n)***. The feature extraction on the edge fully utilizes the edge computing capacity, and it reduces the space complexity on the cloud from ***O(n)*** to ***O(1)*** for one edge node, which greatly saves cloud resources.

After the 32 features (i.e., 9 time-domain features and 7 frequency-domain features from horizontal and vertical vibrations) are calculated, a feature-based Tiny ML model is employed to classify the features against different motor faulty types. The model is a Neural Network (NN), which is designed with 32 input features, two hidden layers with 64 neurons in each layer, and an output layer that corresponds to 5 faulty types. This NN is pre-trained on the cloud using Google's TensorFlow Lite framework, an ML framework that has been optimized for mobile devices. It only provides on-device inference abilities [26], implying that model training and exporting should be conducted off-board.

To further streamline the model and accelerate the feature extraction and model inference processes, valuable features are supposed to be selected as the model input. According to our previous work [27], 10 features including AF, MF, SFW, FK, RMS, SD, and RMSR from horizontal vibration and FK, AF, and CV from vertical vibration, contain almost all the necessary information required for detecting motor faults, leading to a result that is close to that produced by the full 32-feature model. This simplified model achieves a 60% reduction in size compared to the 32-feature model.

C. Device and cloud network layer

This layer enables the communication between the edge device and the cloud focusing solely on wireless connectivity to fit in the Industrial Internet of Things (IIoT) application

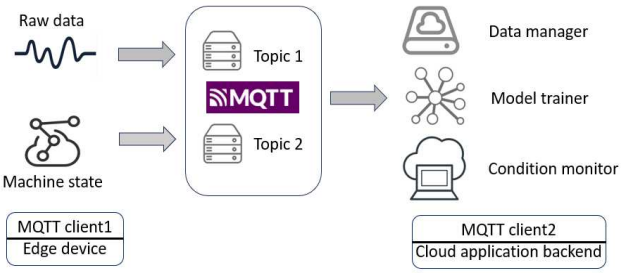


Fig. 4. Communication between the edge device, cloud, and the application through MQTT protocols.

scenarios. The Message Queuing Telemetry Transport 5.0 (MQTT 5.0) communication protocol is adopted to support a quick connection between the edge and the cloud, for its lightweight nature and bandwidth efficiency, which are well-suited for shop floor environments with limited network bandwidth [28]. This protocol allows MQTT clients to publish messages and subscribe to messages from different topics on the server, greatly facilitating the management of various types of data received from distinct edge devices.

Fig.4 presents the connection between the edge device, the cloud, and the cloud application. A connection driver is embedded in the edge device application layer, to connect the MCU to the MQTT server. Given that two different types of data, raw data and features, are to be transmitted, there are two separate topics on the cloud to receive and store the data respectively. The cloud backend receives the data by subscribing to topics on the server and then transfers the data to different users.

Authorized licensed use limited to: FUNDACAO UNIVERSIDADE DO RIO GRANDE. Downloaded on June 26,2026 at 15:54:37 UTC from IEEE Xplore. Restrictions apply.

D. Cloud application layer

The cloud application layer acts as an interface for users, and it can include a variety of components that are utilized for data analysis, decision-making, device management, etc. In this study, three modules are applied: a data manager, a model trainer, and a condition viewer, as illustrated in Fig.4. These are programmed using high-level programming languages such as Python and JavaScript, providing a straightforward setting and easy access for users. The data manager retrieves the data from the cloud via the cloud application backend and saves it on a local computer. It also offers a customizable sampling interface that allows the setting of parameters such as sampling time, sampling rate, local file location, etc. The model trainer is utilized to train the feature-based NN and to convert it into a TensorFlow Lite model suitable for on-device execution. The condition viewer identifies the machine state and the likelihood of specific conditions based on the results from edge inference and displays the machine condition and the prognosis probability.

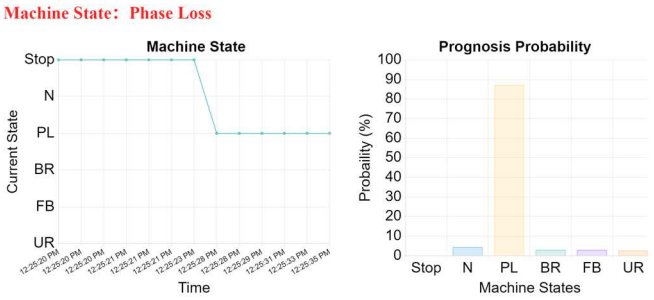


Fig. 5. Illustration of user interface on the cloud.

The system can recognize five operating conditions: normal (N), phase loss (PL), bowed rotor (BR), faulted bearings (FB), and unbalanced rotor (UR), as shown in Fig. 5.

III. Experimental Evaluation

This section performs two experiments to evaluate the performance of the developed edge device on an induction motor for data acquisition and transmission of high-frequency signals and fault diagnosis on the edge. As shown in Fig.6, the edge device is magnetically mounted on the top of a SpectraQuest Machinery Fault Simulator, which is run at three rotating speeds: 1200, 2000, and 3000rpm, under five operating conditions introduced in Section II.D. The normal condition is simulated by a standard motor, while the phase loss is simulated by integrating a phase controller with the operational motor. The abnormalities related to bowed rotor, faulted bearings, and unbalanced rotor are manually introduced into respective motors (e.g., cracks on bearings).

A. Data acquisition and transmission

In this experiment, the horizontal and vertical vibration signal datasets collected by the edge device are compared with those from in-situ commercial sensors, to assess the sensing accuracy and fidelity of the edge device.

First, by comparing the vibration waveforms and frequency spectrums of the signal collected by the device with the signal collected by the commercial sensor, the performance of the device can be checked if these two sources of signals share similar patterns in both the time domain and frequency domain (e.g., the shape of waveforms, dominant frequency). The performance is then further evaluated by checking if an ML classifier can achieve comparable performance on the two sources of signals. The comparable

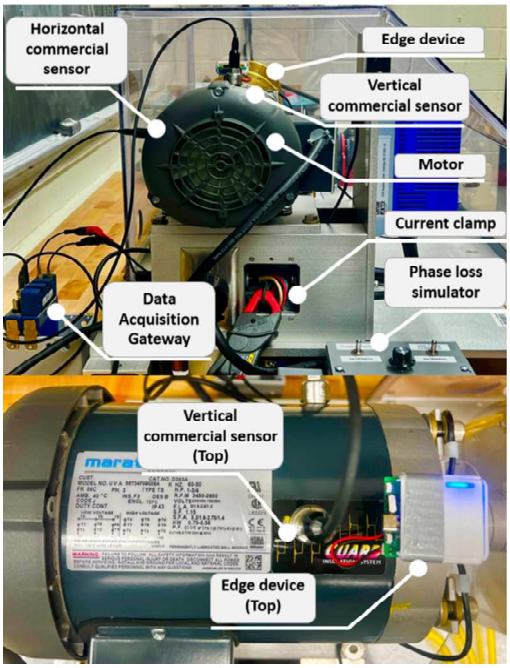


Fig. 6. The left and top view of the experiment setup. Two commercial sensors are connected to the gateway through wires. The edge device is magnetically mounted on the top and wirelessly connected to the server.

classification results would suggest that the edge device performs equivalent to the commercial sensors, effectively capturing the critical vibration signatures that reveal various motor conditions.

B. Edge computing

This experiment is conducted to evaluate the performance of real-time fault diagnosis on the edge. The process consists of two steps: training and validation of the Tiny ML model that is trained on the cloud, and the execution of Tiny ML model inference on the edge device.

In the first step, vibration data collected from five different motor conditions and three rotating speeds are segmented into 1024-sample windows with 50% overlapping between windows. The edge device extracts the time-domain and frequency-domain features from each window and sends the features to the cloud. The NN, with a structure introduced in Section II, is trained on the feature datasets. To further streamline the model, a NN model that takes the 10 most valuable features is trained. A high accuracy rate of these classification models would prove not only the effectiveness of the features extracted by the edge device but also the fidelity of the feature-based classification model.

Following the model training, the NN model is converted into a TensorFlow Lite format that is suitable for deployment on low-power edge devices and downloaded to the edge device for execution. The edge device, now operating in edge computing mode, performs real-time inference using this model and transmits the diagnostic results back to a server for validation. This step involves testing the device across the same five motor conditions previously used for training. The results of the edge inferences are compared with the known conditions of the motor to evaluate the model's precision on edge fault diagnosis.

Authorized licensed use limited to: FUNDACAO UNIVERSIDADE DO RIO GRANDE. Downloaded on June 26,2026 at 15:54:37 UTC from IEEE Xplore. Restrictions apply.

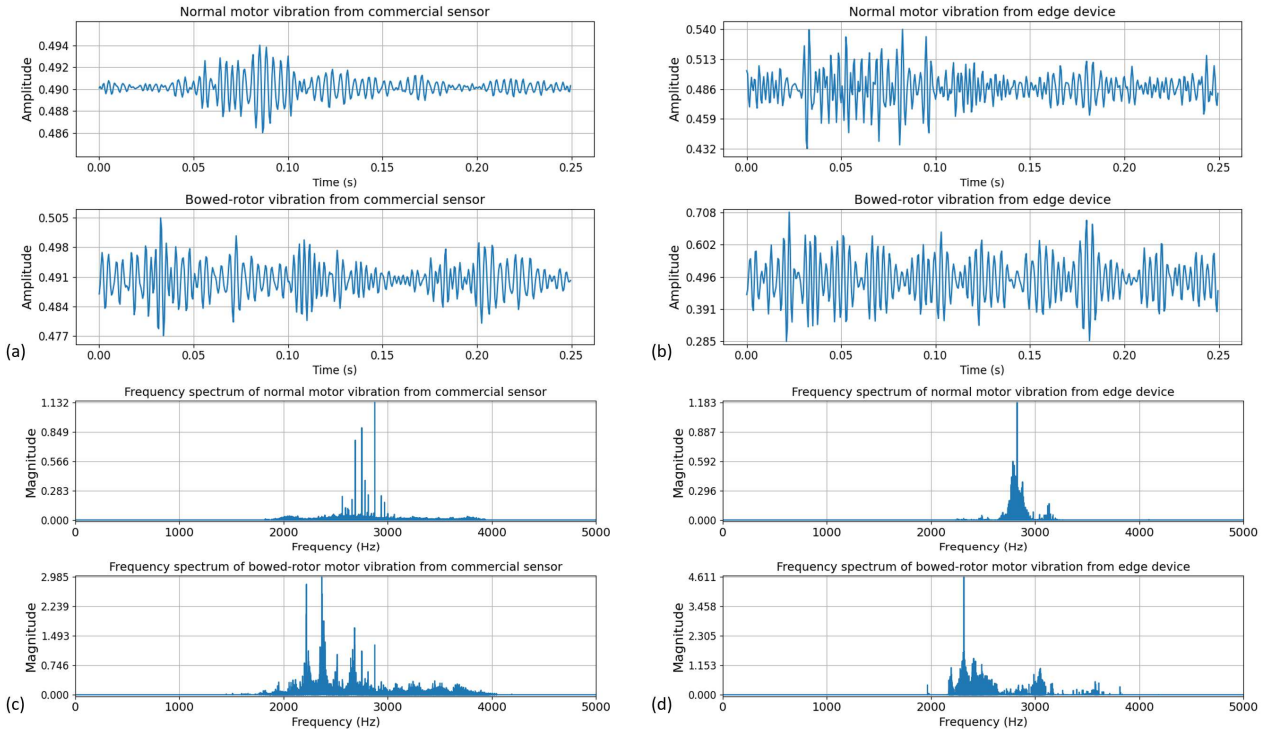


Fig. 7. Comparison between the commercial vibration sensor (a) (c) and the developed edge device (b) (d) of monitoring normal and bowed-rotor motor in time domain and frequency domain.

IV. Results and Discussions

A. Performance of data acquisition and transmission

The waveforms of the normal motor and one with a bowed rotor, running at 2000 RPM, captured by the edge device and the commercial sensor, are depicted in Fig.7(a) and (b). Their corresponding frequency spectrum is presented in Fig.7(c) and (d). These representations visually demonstrate similar shapes of waveforms and patterns in the frequency domain (e.g., dominant frequency, bandwidth), confirming the edge device's capability to capture and transmit the data accurately.

Furthermore, two ML classifiers trained using the data from two different sources, the edge device and the commercial sensor, achieve accuracies of 96.4±0.9% and 91±1.4%, respectively. Remarkably, the model trained with the device data exhibited a 5.4% higher accuracy than its counterpart, demonstrating higher fidelity of the data. This comparison indicates the edge device outperforms the data sampling attributed to its dual-core algorithm which more accurately samples and transmits the original vibration signal.

B. Performance of edge fault diagnosis based on TinyML

During the model training process, an 80/20 split ratio is used in creating training and validation data subsets, and 5-fold cross-validation is utilized, providing a reliable evaluation of the model's performance. As shown in Table 1, a 32-feature-based classification model is applied, achieving a 93.6% accuracy in identifying 5 motor conditions. To improve computational performance and reduce power consumption, the model that takes the 10 most valuable features achieves a 91.2% classification accuracy, which is only slightly lower than the full-feature model. This result suggests that the streamlined feature set can still provide sufficient information for reliable classification, which is advantageous for edge computing scenarios where computational resources are limited. Because a high-speed motor, i.e., running above 2000 RPM, is not commonly used on the shop floor, the 10-feature model is further evaluated under lower RPMs. A 95.7% accuracy is achieved by the edge device.

To further test the model's robustness, it is deployed to detect motor faults using the data sampled from actual normal and abnormal motors at low-speed mode (i.e., 1200 and 2000 RPMs) and all-speed mode (i.e., 1200, 2000, and 3000 RPMs) settings. The confusion matrices of diagnosis are shown in Fig. 8. Because the phase loss fault, which occurs when one phase of the three-phase power supply to the motor is lost, exhibits less correlation in vibration signature, especially at high RPMs, the accuracy in differentiating it from the normal condition is lower than other fault conditions.

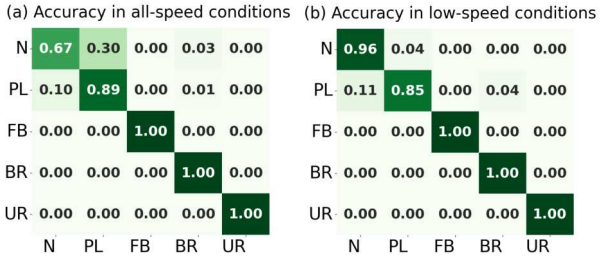


Fig. 8. Diagnostic accuracy of each type of fault using Tiny ML under different rotating speed conditions.

TABLE I Comparison between different data sources and Tiny ML models in motor fault diagnosis accuracy under various motor RPMs

Data source	Motor RPM	Model	Accuracy (%)
Edge device	1200, 2000, 3000	NN with 32 features	93.6 ± 1.7
Commercial sensor	1200, 2000, 3000	NN with 32 features	72.8 ± 2.9
Edge device	1200, 2000, 3000	NN with 10 features	91.2 ± 1.4
Edge device	1200, 2000	NN with 10 features	95.7 ± 0.4
Commercial sensor	1200, 2000	NN with 10 features	90.5 ± 1.1

Authorized licensed use limited to: FUNDACAO UNIVERSIDADE DO RIO GRANDE. Downloaded on June 26,2026 at 15:54:37 UTC from IEEE Xplore. Restrictions apply.

Given the constraint of the window size set at 1024 with a 50% overlapping rate, equivalent to a duration of 0.05 seconds, there is a potential concern that some critical fault-related vibration signatures might not manifest within such a brief time. As a result, a reputation-based decision method is implemented. This method groups 15 successive windows and performs an aggregation to determine the classification result. This approach significantly enhanced the end decision accuracy, attaining a perfect accuracy of 100% on the 5 types of motors under 2000 RPM.

Additionally, power consumption is a limiting factor for battery-powered remote devices. To achieve condition monitoring over a long period, a one-hour hibernation mode is adopted on the device. In this mode, the device waking up detects the anomalies once an hour and then transmits the inferring result to the server before the next hibernation. The detecting time, voltage, and current are measured during the inferring process to calculate the maximum continuous working time after a full charge. The power consumption of the edge device under different working modes is estimated in Table II. The edge device can last for 27 days after the equipped 500 mAh lithium battery is fully charged.

TABLE II THE POWER CONSUMPTION IN THE ONE-HOUR CONDITION MONITORING SCENARIO

Task on device	Current (mA)	Voltage (V)	Duration (s)	Energy (mJ)
Wake-up & Connection	95.3	3.3	3.5	1100.7
Edge fault diagnosis	75.9	3.3	5.7	1427.7
Deep sleep	0.02	3.3	3590	236.9

V. CONCLUSION

This paper presented a solution for industrial machine condition monitoring that is not only cost-effective but also deployable and reliable, consisting of an innovative prototype of an edge device and an end-to-end system. As the core of this solution, the edge device can handle the precise collection and real-time transmission of high-frequency vibration data, as well as on-device fault diagnosis by integration with Tiny ML. Future work would explore its reliability in real plants by integration with federated learning, which enables the edge devices to learn collaboratively while keeping the data localized, thereby enhancing the model's accuracy and generalizability across various industrial environments.

ACKNOWLEDGMENT

The work is supported by the National Science Foundation under grant No. 2015889.

REFERENCES

[1] Z. Shakeri and K. Benfriha, "An Overview on Smart Scheduling in MES: In the Context of Industry 4.0," *27th International Conference on Automation and Computing*, 2022.

[2] A. Jamwal, R. Agrawal, M. Sharma, and A. Giallanza, "Industry 4.0 technologies for manufacturing sustainability: A systematic review and future research directions," *Applied Sciences (Switzerland)*, vol. 11, no. 12. MDPI AG, Jun. 02, 2021.

[3] M. Gopala and K. Sriram, "Edge computing vs. Cloud computing: an overview of big data challenges and opportunities for large enterprises", *International Research Journal of Modernization in Engineering Technology and Science,* vol. 4, No. 1, Jan 2022.

[4] H. Hua, Y. Li, T. Wang, N. Dong, W. Li, and J. Cao, "Edge Computing with Artificial Intelligence: A Machine Learning Perspective," *ACM Comput Surv*, vol. 55, no. 9, Jan. 2023.

[5] L. Lan, R. Shi, B. Wang, L. Zhang, and N. Jiang, "A universal complex event processing mechanism based on edge computing for IoT real-time monitoring," *IEEE Access*, vol. 7, pp. 101865–101878, 2019.

[6] Y. C. Liang, W. D. Li, X. Lu, and S. Wang, "Fog computing and convolutional neural network enabled prognosis for machining process optimization," *J Manuf Syst*, vol. 52, pp. 32–42, Jul. 2019.

[7] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge Computing in Industrial Internet of Things: Architecture, Advances and Challenges," *IEEE Communications Surveys and Tutorials*, vol. 22, no. 4, pp. 2462–2488, Oct. 2020.

[8] S. Lu, J. Lu, K. An, X. Wang, and Q. He, "Edge Computing on IoT for Machine Signal Processing and Fault Diagnosis: A Review," *IEEE Internet of Things Journal*, vol. 10, no. 13. Institute of Electrical and Electronics Engineers Inc., pp. 11093–11116, Jul. 01, 2023.

[9] S. Aiassa, S. Carrara, and D. Demarchi, "Optimized Sampling Rate for Voltammetry-Based Electrochemical Sensing in Wearable and IoT Applications," *IEEE Sensors Letters*, vol. 3, no. 6, pp. 1-4, June 2019,

[10] H. Tang, S. Lu, G. Qian, J. Ding, Y. Liu, and Q. Wang, "IoT-Based Signal Enhancement & Compression Method for Efficient Motor Bearing Diagnosis," *IEEE Sens J*, vol. 21, no. 2, pp. 1820–1828, 2021.

[11] X. Wang, S. Lu, W. Huang, Q. Wang, S. Zhang, and M. Xia, "Efficient Data Reduction at the Edge of Industrial Internet of Things for PMSM Bearing Fault Diagnosis," *IEEE Trans Instrum Meas*, vol. 70, 2021.

[12] T. T. Huong *et al.*, "Federated Learning-Based Explainable Anomaly Detection for Industrial Control Systems," *IEEE Access*, vol. 10, pp. 53854–53872, 2022.

[13] H. Huang, L. Yang, Y. Wang, X. Xu, and Y. Lu, "Digital Twin-driven online anomaly detection for an automation system based on edge intelligence," *J Manuf Syst*, vol. 59, pp. 138–150, 2021.

[14] S. Zhou, Y. Du, B. Chen, Y. Li, and X. Luan, "An Intelligent IoT Sensing System for Rail Vehicle Running States Based on TinyML," *IEEE Access*, vol. 10, pp. 98860–98871, 2022.

[15] Q. N. Minh, V. H. Nguyen, V. K. Quy, L. A. Ngoc, A. Chehri, and G. Jeon, "Edge Computing for IoT-Enabled Smart Grid: The Future of Energy," *Energies (Basel)*, vol. 15, no. 17, Sep. 2022.

[16] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, and R. X. Gao, "Deep learning and its applications to machine health monitoring," *Mechanical Systems and Signal Processing*, vol. 115. Academic Press, pp. 213–237, Jan. 15, 2019.

[17] Z. Chen, Y. Gao, and J. Liang, "LOPdM: A Low-power On-device Predictive Maintenance System Based on Self-powered Sensing and TinyML," *IEEE Trans Instrum Meas*, 2023.

[18] J. de las Morenas, F. Moya-Fernández, and J. A. López-Gómez, "The Edge Application of Machine Learning Techniques for Fault Diagnosis in Electrical Machines," *Sensors*, vol. 23, no. 5, Mar. 2023.

[19] L. Magadán, F. J. Suárez, and D. F. García, "Low-Cost Industrial IoT System for Wireless Monitoring of Electric Motors Condition," *Mobile Networks and Applications*, vol. 28, no. 1, pp. 97–106, Feb. 2023.

[20] D. Mazzei, and G. Fantoni, "A Full Stack for Quick Prototyping of IoT Solutions," *Ann. Telecommun, vol.* 73, pp. 439–449, 2018.

[21] M. H. Mohd Ghazali and W. Rahiman, "Vibration Analysis for Machine Monitoring and Diagnosis: A Systematic Review," *Shock and Vibration*, vol. 2021. Hindawi Limited, 2021.

[22] F. Leens, "An introduction to I2C and SPI protocols," *IEEE Instrumentation & Measurement Magazine*, vol. 12, no. 1, Feb. 2009.

[23] Guan F, Peng L, Perneel L, "Open source FreeRTOS as a case study in real-time operating system evolution," Journal of Systems and Software, vol. 118, pp. 19-35, 2016

[24] Z. Xu, Y. Li, Z. Wang, and J. Xuan, "A Novel Clustering Method Combining ART with Yu's Norm for Fault Diagnosis of Bearings," *Shock and Vibration*, vol. 2016.

[25] S Lennart Johnsson, "Cooley-Tukey FFT on the Connection Machine," *Parallel Computing*, vol.18, issue 11, pp.1201-1221, 1992

[26] J. Chen and X. Ran, "Deep Learning with Edge Computing: A Review," *Proceedings of the IEEE*, vol. 107, no. 8, Aug. 01, 2019.

[27] Y. Wang and P. Wang, "Explainable machine learning for motor fault diagnosis," *IEEE Instru & Measurement Technology Conf.*, 2023.

[28] D. Soni and A. Makwana, "A Survey On Mqtt: A Protocol Of Internet of Things(IoT)," *International conference on telecommunication, power analysis and computing techniques*, Vol. 20. 2017.

Authorized licensed use limited to: FUNDACAO UNIVERSIDADE DO RIO GRANDE. Downloaded on June 26,2026 at 15:54:37 UTC from IEEE Xplore. Restrictions apply.